human

Show me buildings in Exeter that are within 150m of a river.

host_agent

Calling tool send_message with args : {'target': 'planning_agent', 'task_description': 'User wants to see buildings in Exeter that are within 150m of a river. Provide the general steps to solve this geospatial query.'}

planning_agent

The area is Exeter, so there will be 1 entry for Exeter. The entities to search for are buildings in Exeter, and we need to find as many entries as possible. Additionally, we need to identify the river in Exeter and apply the condition of being within 150m of that river.

output steps: ["Find Exeter 1 area", "Find buildings in Exeter as many search results", "Find rivers in Exeter 1 or more results", "Apply condition of buildings within 150m of the river"]

host_agent

Calling tool send_message with args : {'target': 'named_area', 'task_description': 'Find the area polygon for Exeter.'}

named_area_agent

Calling tool call_os_ngd with args : {'filters': None, 'bbox': None, 'polygon_or_point': True, 'street_address': None, 'filename': 'exeter_area_polygon'} Tool call_os_ngd returned result : ['Search results have been found. \n Artifact generated is : exeter_area_polygon,\n Description is : A geopandas dataframe containing named area data with no filters applied for the query (so remember no search is performed here so spurious entities present) Find the area polygon for Exeter..,\n count of records fetched : 98.\n ',] Calling tool send_message with args : {'target': 'data_analysis_agent', 'task_description': "Filter the artifact 'exeter_area_polygon' to find the specific polygon for Exeter. Save the filtered results as 'filtered_exeter_area_polygon'."}

data_analysis_agent

Tool Code Executor

```
def filter_exeter_area_polygon(data: list):
    gdf = data[0]
    # Filter for polygons specifically representing Exeter
    # Use name1_text for 'Exeter', and also check for relevant descriptiongroup/description
    filtered = gdf[
        (
            (gdf['name1_text'].str.lower() == 'exeter') |
            (gdf['name1_text'].str.lower().str.contains('exeter'))
        ) & (
            (gdf['descriptiongroup'].isin(['Settlement', 'Part Of Settlement'])) |
            (gdf['description'].isin(['City', 'Town', 'Suburban Area']))
        )
    ]
    summary = f"Filtered polygons for Exeter. Number of matching polygons: {len(filtered)}."
    artifact_name = 'filtered_exeter_area_polygon'
    artifact_description = "GeoDataFrame containing only the polygon(s) representing Exeter, filtered by name and settlement/city/town/suburban area descriptio
    return [summary, artifact_name, artifact_description, filtered]

filter_exeter_area_polygon
```

Filtered polygons for Exeter. Number of matching polygons: 1. An artifact has been generated with the name: filtered_exeter_area_polygon and the description: GeoDataFrame containing only the polygon(s) representing Exeter, filtered by name and settlement/city/town/suburban area description.

named_area_agent

The area polygon for Exeter has been successfully filtered. The artifact name is `filtered_exeter_area_polygon`, which contains the specific polygon representing Exeter.

host_agent

Calling tool send_message with args : {'target': 'buildings', 'task_description': "Find all buildings within the area defined by the artifact 'filtered_exeter_area_polygon'."}

buildings_agent

Calling tool call_os_ngd with args : {'filters': None, 'bbox': 'filtered_exeter_area_polygon', 'polygon_or_point': True, 'street_address': None, 'filename': 'all_buildings_in_filtered_exeter_area'} Tool call_os_ngd returned result : ['Multiple search results have been found in multiple datasets. A summary of each is provided.\n Artifacts generated are : [\'buildings_all_buildings_in_filtered_exeter_area\', \'buildingline_all_buildings_in_filtered_exeter_area\',

\'buildingpart_all_buildings_in_filtered_exeter_area\'],\n Descriptions are : ["A geopandas dataframe containing building data with filters and bbox applied for the query Find all buildings within the area defined by the artifact 'filtered_exeter_area_polygon'. using filters None.", "A geopandas dataframe containing building line data with filters and bbox applied for the query Find all buildings within the area defined by the artifact 'filtered_exeter_area_polygon'. using filters None.", "A geopandas dataframe containing building part data with filters and bbox applied for the query Find all buildings within the area defined by the artifact 'filtered_exeter_area_polygon'. using filters None."],\n counts of records fetched are : [69169, 69342, 85480].\n ', [, ,]] All buildings within the area defined by the artifact 'filtered_exeter_area_polygon' have been found. Here are the relevant data artifacts generated:

- buildings_all_buildings_in_filtered_exeter_area: Contains 69,169 records of building data within the specified area.
- buildingline_all_buildings_in_filtered_exeter_area: Contains 69,342 records of building line data.
- buildingpart_all_buildings_in_filtered_exeter_area: Contains 85,480 records of building part data.

If you need further filtering or analysis on these datasets, please specify your requirements.

host_agent

Calling tool send_message with args : {'target': 'water_network', 'task_description': "Find all rivers within the area defined by the artifact 'filtered_exeter_area_polygon'."}

water_network_agent

Calling tool call_os_ngd with args : {'filters': ['rivers'], 'bbox': 'filtered_exeter_area_polygon', 'polygon_or_point': True, 'street_address': False, 'filename': 'exeter_rivers'} Tool call_os_ngd returned result : ['Multiple search results have been found in multiple datasets. A summary of each is provided.\n Artifacts generated are : [\'waterlinkset_exeter_rivers\', \'waterlink_exeter_rivers\'],\n Descriptions are : ["A geopandas dataframe containing water link set data with bbox applied for the query Find all rivers within the area defined by the artifact 'filtered_exeter_area_polygon'. with no filters applied for the query (so remember no search is performed here so spurious entities present).", "A geopandas dataframe containing water link data with bbox applied for the query Find all rivers within the area defined by the artifact 'filtered_exeter_area_polygon'. with no filters applied for the query (so remember no search is performed here so spurious entities present)."],\n counts of records fetched are : [11, 430].\n ', [,]] Calling tool send_message with args : {'target': 'data_analysis_agent', 'task_description': "Analyze the artifact 'waterlinkset_exeter_rivers' to filter out only the river data. Save the results as 'filtered_rivers_waterlinkset'."}

data_analysis_agent

Tool Code Executor

```
def filter_river_data(data: list):
    import geopandas as gpd
    # The relevant columns for filtering river data are 'description' and 'name1_text'.
    gdf = data[0]
    # Possible river-related descriptions and names
    river_keywords = ['River', 'Brook', 'Stream', 'Leat', 'Watercourse']
    # Filter rows where 'description' or 'name1_text' indicate a river or similar watercourse
    mask = (
        gdf['description'].str.contains('Watercourse', case=False, na=False) |
        gdf['description'].str.contains('Stretch Of Inland Water', case=False, na=False) |
        gdf['name1_text'].str.contains('River', case=False, na=False) |
        gdf['name1_text'].str.contains('Brook', case=False, na=False) |
        gdf['name1_text'].str.contains('Stream', case=False, na=False) |
        gdf['name1_text'].str.contains('Leat', case=False, na=False)
    )
    filtered = gdf[mask].copy()
    summary = f"Filtered river-related features: {len(filtered)} out of {len(gdf)} total entries."
    artifact_name = 'filtered_rivers_waterlinkset'
    artifact_description = 'GeoDataFrame containing only river, brook, stream, leat, and watercourse features from the original waterlinkset_exeter_rivers.'
    return [summary, artifact_name, artifact_description, filtered]
```

Filtered river-related features: 10 out of 11 total entries. An artifact has been generated with the name 'filtered_rivers_waterlinkset' and contains a GeoDataFrame with only river, brook, stream, leat, and watercourse features from the original 'waterlinkset_exeter_rivers' dataset. If you need further analysis or details on this filtered data, please let me know!

Calling tool send_message with args : {'target': 'data_analysis_agent', 'task_description': "Analyze the artifact 'waterlink_exeter_rivers' to filter out only the river data. Save the results as 'filtered_rivers_waterlink'."} Tool Code Executor

```
def filter_river_data(data: list):
    import geopandas as gpd
    gdf = data[0]
    river_keywords = ['river', 'brook', 'stream', 'leat', 'watercourse']
    mask_description = gdf['description'].str.lower().isin(river_keywords)
    mask_name = gdf['name1_text'].str.lower().str.contains('|'.join(river_keywords), na=False)
    mask = mask_description | mask_name
    filtered = gdf[mask]
    summary = f"Filtered river-related features: {len(filtered)} out of {len(gdf)} total entries."
    artifact_name = 'filtered_rivers_waterlink'
    artifact_description = "GeoDataFrame containing only river, brook, stream, leat, and watercourse features from the original waterlink_exeter_rivers."
    return [summary, artifact_name, artifact_description, filtered]
```

Filtered river-related features: 341 out of 430 total entries. An artifact has been generated with the name 'filtered_rivers_waterlink' and contains a GeoDataFrame with only river, brook, stream, leat, and watercourse features from the original 'waterlink_exeter_rivers' dataset. If you need further analysis or details on this filtered data, please let me know!

water_network_agent

Here are the filtered artifacts containing river data within the 'filtered_exeter_area_polygon':

1. **Filtered Rivers Waterlinkset**: Contains 10 river-related features.
2. Artifact Name: `filtered_rivers_waterlinkset`

   Description: GeoDataFrame with only river, brook, stream, leat, and watercourse features.

   Filtered Rivers Waterlink: Contains 341 river-related features.

5. Artifact Name: `filtered_rivers_waterlink`
6. Description: GeoDataFrame with only river, brook, stream, leat, and watercourse features.

If you need further analysis or details, feel free to ask!

host_agent

Calling tool send_message with args : {'target': 'plotting_agent', 'task_description': "Plot all buildings from 'buildings_all_buildings_in_filtered_exeter_area' that are within 150m of any river from 'filtered_rivers_waterlink' in Exeter. Show the spatial relationship and highlight the buildings that meet the distance condition."}

plotting_agent

Tool Code Executor

```
def plot_buildings_within_150m_of_rivers(data: list):
    import geopandas as gpd
    import folium
    from shapely.geometry import Polygon, LineString
    from shapely.ops import unary_union
    import os
```

```
    # Unpack data
    buildings = data[0].copy()
    rivers = data[1].copy()

    # Ensure CRS is projected for distance calculation, then back to EPSG:4326 for folium
    if buildings.crs is None:
        buildings.set_crs(epsg=27700, inplace=True)  # Assume British National Grid if missing
    if rivers.crs is None:
        rivers.set_crs(epsg=27700, inplace=True)

    # Project to British National Grid for accurate distance calculations
    buildings = buildings.to_crs(epsg=27700)
    rivers = rivers.to_crs(epsg=27700)

    # Create a 150m buffer around all river geometries
    river_buffer = rivers.buffer(150)
    river_buffer_union = gpd.GeoSeries(unary_union(river_buffer), crs=27700)

    # Find buildings that intersect with the buffer
    buildings_within_150m = buildings[buildings.intersects(river_buffer_union.iloc[0])]

    # Prepare for plotting: convert all to EPSG:4326
    buildings = buildings.to_crs(epsg=4326)
    rivers = rivers.to_crs(epsg=4326)
    river_buffer_union = river_buffer_union.to_crs(epsg=4326)
    buildings_within_150m = buildings_within_150m.to_crs(epsg=4326)

    # Create folium map centered on Exeter
    centroid = buildings_within_150m.geometry.unary_union.centroid
    m = folium.Map(location=[centroid.y, centroid.x], zoom_start=13, tiles='cartodbpositron')

    # Plot all buildings (grey)
    folium.GeoJson(
        buildings[['geometry']],
        name='All Buildings',
        style_function=lambda x: {'color': 'grey', 'weight': 0.5, 'fillOpacity': 0.1}
    ).add_to(m)

    # Plot rivers (blue)
    folium.GeoJson(
        rivers[['geometry']],
        name='Rivers',
        style_function=lambda x: {'color': 'blue', 'weight': 2}
    ).add_to(m)

    # Plot 150m buffer (light blue)
    folium.GeoJson(
```

```
        river_buffer_union,
        name='150m River Buffer',
        style_function=lambda x: {'color': 'lightblue', 'weight': 1, 'fillOpacity': 0.15}
    ).add_to(m)

    # Plot highlighted buildings within 150m (red)
    folium.GeoJson(
        buildings_within_150m[['geometry']],
        name='Buildings within 150m of River',
        style_function=lambda x: {'color': 'red', 'weight': 1, 'fillOpacity': 0.5}
    ).add_to(m)

    folium.LayerControl().add_to(m)

    # Save map
    map_filename = 'buildings_within_150m_of_rivers_exeter.html'
    m.save(map_filename)

    # Prepare summary
    summary_rows = buildings_within_150m[['osid', 'description', 'buildinguse', 'geometry_area_m2']].head(5)
    summary = f"There are {len(buildings_within_150m)} buildings within 150m of any river in Exeter. First 5: {summary_rows.to_dict(orient='records')}"
    artifact_name = 'buildings_within_150m_of_rivers_map'
    artifact_description = 'Folium map showing all buildings, rivers, 150m buffer, and buildings within 150m of any river in Exeter. Highlighted buildings are
    return [summary, artifact_name, artifact_description, map_filename]
```

There are 17530 buildings within 150m of any river in Exeter. First 5: [{'osid': '5b8bc540-b5d9-47bb-87bb-0d1c0a69bade', 'description': 'Detached House', 'buildinguse': 'Residential Accommodation', 'geometry_area_m2': 63.966}, {'osid': '90b4bc37-bec6-433c-a237-a024bafb8457', 'description': 'Domestic Outbuilding', 'buildinguse': 'Unknown', 'geometry_area_m2': 21.506}, {'osid': 'd6dddb8a-0e78-4b45-a739-22b1a84ece8d', 'description': 'Unknown Building', 'buildinguse': 'Unknown', 'geometry_area_m2': 33.799}, {'osid': '35bbf17e-145e-4a6f-9a90-6fa36ee54065', 'description': 'Unknown Building', 'buildinguse': 'Unknown', 'geometry_area_m2': 3.16}, {'osid': '89e65c92-6b04-4ee5-b4c4-d67398374622', 'description': 'Unknown Building', 'buildinguse': 'Unknown', 'geometry_area_m2': 2.897}]

There are 17,530 buildings within 150 meters of any river in Exeter. The first 5 buildings that meet this condition are:

1. Detached House (osid: 5b8bc540-b5d9-47bb-87bb-0d1c0a69bade, area: 63.97 m², use: Residential Accommodation)
2. Domestic Outbuilding (osid: 90b4bc37-bec6-433c-a237-a024bafb8457, area: 21.51 m², use: Unknown)
3. Unknown Building (osid: d6dddb8a-0e78-4b45-a739-22b1a84ece8d, area: 33.80 m², use: Unknown)
4. Unknown Building (osid: 35bbf17e-145e-4a6f-9a90-6fa36ee54065, area: 3.16 m², use: Unknown)
5. Unknown Building (osid: 89e65c92-6b04-4ee5-b4c4-d67398374622, area: 2.90 m², use: Unknown)

An interactive folium map has been generated showing: - All buildings (grey) - All rivers (blue) - The 150m buffer around rivers (light blue) - Buildings within 150m of any river (highlighted in red)

Artifact name: buildings_within_150m_of_rivers_map
Artifact description: Folium map showing all buildings, rivers, 150m buffer, and buildings within 150m of any river in Exeter. Highlighted buildings are within 150m.
Artifact data: buildings_within_150m_of_rivers_exeter.html

host_agent

There are 17,530 buildings in Exeter that are within 150 meters of a river. Here’s what was done:

- All buildings in Exeter were identified.
- All rivers in Exeter were identified.
- A spatial analysis was performed to find which buildings are within 150 meters of any river.

A map has been generated showing: - All buildings (grey) - All rivers (blue) - The 150m buffer around rivers (light blue) - Buildings within 150m of any river (highlighted in red)

If you’d like to view or download the interactive map, let me know! If you need details about specific buildings or want to refine the search, just ask.